

Beijing Institute of Technology (BIT)

Netflix stock price prediction

School of Computer Science

狄苏菲 伯汀娜

2024/11/1

Outline

1 Introduction	1
2 Problem definition	2
3 Main content	2
3.1 Plan	2
3.2 Algorithm	3
3.2.1 Data Overview	3
3.2.2 Data Preparation	5
3.2.3 Linear Regression	6
3.2.4 Evaluation of the model	6
3.2.5 Visualization of the model	7
3.2.6 Making a prediction	9
3.2.7 Saving the data as CSV	9
3.2.8 Saved result	10
3.2.9 Resulting graph	11
3.3 Code	12
4 Conclusion	14

1 Introduction

Predicting Netflix stock prices is a project driven by a desire to empower investors with more confidence in their decisions. By leveraging data and predictive models, the goal is to provide insights into potential future price movements, enabling investors to make more informed choices. Accurate predictions could lead to increased profitability by identifying opportunities to buy low and sell high, or to sell before potential price drops.

However, predicting stock prices is notoriously difficult. Stock prices are highly volatile and influenced by a multitude of factors, including economic conditions, industry trends, company performance, and news events. Even sophisticated models struggle to achieve consistent accuracy, facing challenges like overfitting to past data, limited data availability, and the inherent randomness of the market.

To tackle these challenges, careful model selection, feature engineering, and data preparation are crucial. Thorough testing on historical data is essential before using the model for real-world predictions. So, while predicting Netflix stock prices is a challenging endeavor, it still holds the potential to provide valuable insights for investors.

2 Problem definition

In today's digital age, computer technologies play significant role in our daily lives, shaping how we communicate, work, and make financial decisions. The stock market, in particular, has been transformed by advanced algorithms, real-time data analysis, and machine learning techniques. The popularity of stock trading has surged in recent years, driven by the accessibility of online trading platforms and the growing interest in personal finance. As more people seek to build wealth through investments, understanding stock price movements becomes crucial.

Netflix, a leading streaming service provider, has shown significant fluctuations in its stock price influenced by various factors including subscriber growth, market trends, and overall economic conditions. Accurate predictions of stock prices can help investors make informed decisions and minimize financial risks.

3 Main content

3.1 Plan

To model and predict Netflix stock prices, various machine learning techniques can be employed. Key steps include:

- Data collection using libraries to fetch historical stock data;

- Analyzing factors influencing stock prices, such as market trends;
- Implementing algorithms and evaluating performance using metrics.

In this project we built our model using linear regression method, we were also using NumPy, Pandas, Matplotlib and Scikit-learn libraries of the Python language.

3.2 Algorithm

3.2.1 Data Overview

1) Importing Libraries:

- **NumPy** is used for working with arrays, matrices, and mathematical operations;
- **Pandas** is used for data manipulation, providing data structures like DataFrames for convenient data handling and analysis;
- **Matplotlib** is used for creating various types of plots and visualizing data;
- By using **Scikit-learn library** it's easier to split the dataset, to create a linear regression model and count various metrics for evaluating the performance

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (mean_squared_error, mean_absolute_error,
                             mean_absolute_percentage_error, r2_score)
```

2) Data Loading:

In this step we are loading the data from a CSV file into a Pandas DataFrame and getting basic information about the dataset and its structure. After that we are creating a deep copy so the original dataset will not be effected.

```
# Load the dataset
df = pd.read_csv("dataset.csv")
# Check dataset information
print(df.info())
# Check top five rows
print(df.head())
# Check bottom five rows
print(df.tail())
# Create deep copy object (the original object will not be effected)
viz = df.copy()
```

Here is the info of the dataset. The data is derived from the dataset of Netflix Stock Price Prediction by Kaggle. The Dataset contains data for 5 years i.e. from 5th Feb 2018 to 5th Feb 2022. There are 7 columns in our dataset and each of them consists of 1009 records:

- Date (Everyday price)
- Open (Price from the first transaction of a trading day)
- High (Today's high)
- Low (Today's low)
- Close (Close price adjusted for splits)
- Adj (Adjusted close price)
- Volume (Volume of stocks)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1009 entries, 0 to 1008
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        1009 non-null  object
1   Open        1009 non-null  float64
2   High        1009 non-null  float64
3   Low         1009 non-null  float64
4   Close       1009 non-null  float64
5   Adj Close   1009 non-null  float64
6   Volume      1009 non-null  int64
dtypes: float64(5), int64(1), object(1)
memory usage: 55.3+ KB
```

And here are the first 5 records and the last 5 ones of the dataset.

	Date	Open	High	...	Close	Adj Close	Volume
0	2018-02-05	262.000000	267.899994	...	254.259995	254.259995	11896100
1	2018-02-06	247.699997	266.700012	...	265.720001	265.720001	12595800
2	2018-02-07	266.579987	272.450012	...	264.559998	264.559998	8981500
3	2018-02-08	267.079987	267.619995	...	250.100006	250.100006	9306700
4	2018-02-09	253.850006	255.800003	...	249.470001	249.470001	16906900

[5 rows x 7 columns]

	Date	Open	High	...	Close	Adj Close	Volume
1004	2022-01-31	401.970001	427.700012	...	427.140015	427.140015	20047500
1005	2022-02-01	432.959991	458.480011	...	457.130005	457.130005	22542300
1006	2022-02-02	448.250000	451.980011	...	429.480011	429.480011	14346000
1007	2022-02-03	421.440002	429.260010	...	405.600006	405.600006	9905200
1008	2022-02-04	407.309998	412.769989	...	410.170013	410.170013	7782400

[5 rows x 7 columns]

3.2.2 Data Preparation

In this step we found the rows in the dataset that contain missing, wrong or unusable values. Then we decided whether to keep the fields and fill in the missing values or simply delete the fields.

```
# Check missing values
print(df.isna().sum())

# Split the data
x = df.drop(labels=["Close", "Date", "Adj Close"], axis=1)
y = df["Close"]
x_train, x_test, y_train, y_test = train_test_split(*arrays: x, y, random_state=0)
print(x_train.head())
```

On this picture is the code where we:

- checked the dataset for missing values;
- separated features (input variables) and the target variable (closing price);
- split the data into training and testing sets for model training and evaluation.

```
[5 rows x 7 columns]
Date          0
Open          0
High          0
Low           0
Close         0
Adj Close     0
Volume        0
dtype: int64
```

Here we checked the dataset for missing or wrong values. Our dataset was clean.

	Open	High	Low	Volume
810	506.760010	510.480011	503.000000	4388800
2	266.579987	272.450012	264.329987	8981500
481	326.779999	329.859985	325.529999	3806900
477	332.959991	333.820007	326.010010	5036100
308	373.679993	374.579987	369.119995	3821700

Here are the first 5 records of the training dataset.

3.2.3 Linear Regression

Based on the columns in the database we will find the correlation between different factors and define the price. We used a linear regression, a statistical method used to model the relationship between a dependent variable and one or more independent variables. This algorithm aims to find a linear equation that best represents the relationship between these variables. We trained the model and made a prediction on the test dataset.

```
# Linear Regression
model_lnr = LinearRegression()
model_lnr.fit(x_train, y_train)
y_pred = model_lnr.predict(x_test)
```

3.2.4 Evaluation of the model

In this step we are evaluating our model by using different methods.

- **Mean Squared Error (MSE)** calculates the average squared difference between the predicted values and the actual values. Also, we calculate **the square root of the MSE (RMSE)**.
- **Mean Absolute Error (MAE)** calculates the average absolute difference between predicted values and actual values.
- **Mean Absolute Percentage Error (MAPE)** calculates the average percentage difference between predicted values and actual values.
- **R-squared (R2) Score** measures the proportion of variance in the target variable that is explained by the model.

```
# Model Evaluation
print("MSE", round(mean_squared_error(y_test, y_pred), 3))
print("RMSE", round(np.sqrt(mean_squared_error(y_test, y_pred)), 3))
print("MAE", round(mean_absolute_error(y_test, y_pred), 3))
print("MAPE", round(mean_absolute_percentage_error(y_test, y_pred), 3))
print("R2 Score : ", round(r2_score(y_test, y_pred), 3))
```

```
MSE 19.164
RMSE 4.378
MAE 3.124
MAPE 0.008
R2 Score : 0.998
```

The **MSE** is 19.164 which is a relatively low value, indicating that the model's predictions are quite close to the actual values.

MAE means that the model's predictions are about 3.124 units off from the actual values. This is lower than the **RMSE**, so there are not many errors in the model's predictions.

MAPE is indicating that the predictions are about 0.8% off from the actual values.

The **R2 Score** indicates that model explains a 99.8% of the variance in the target variable.

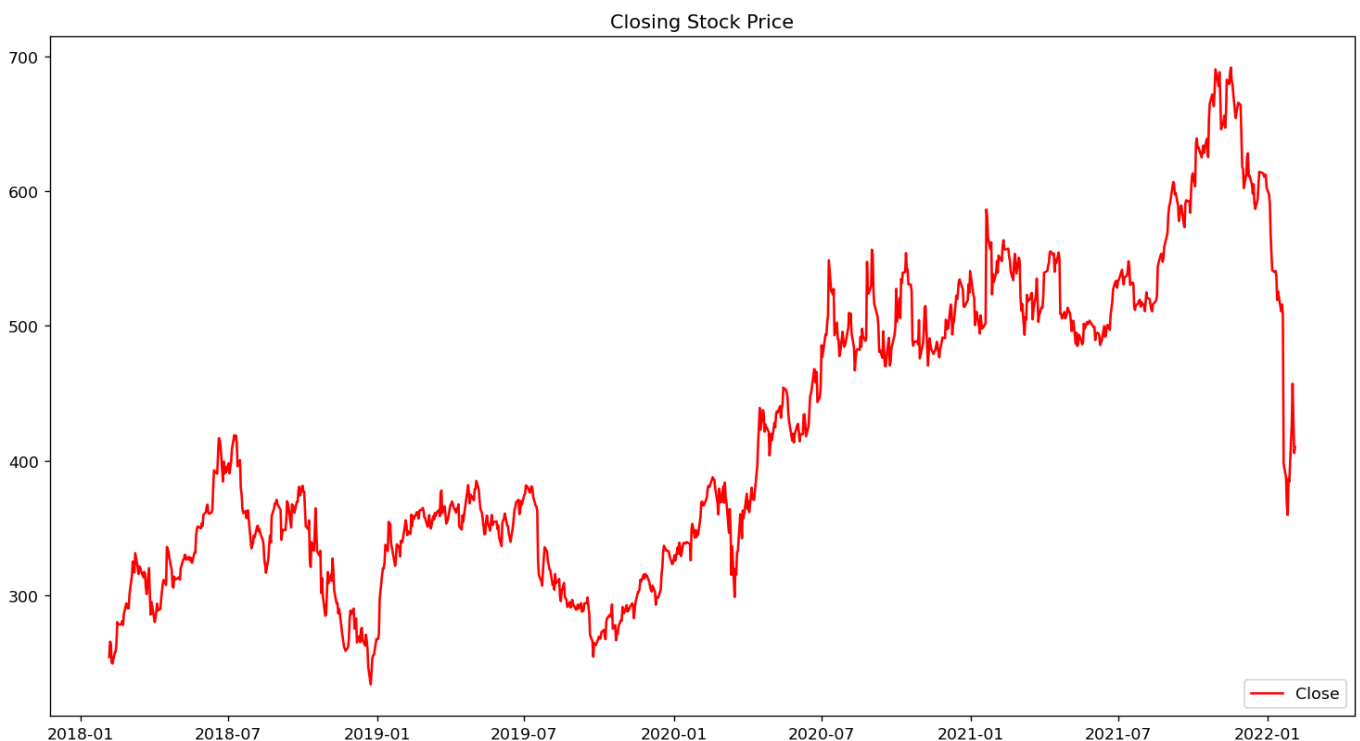
This extremely high score means that the model is a very good fit for the data.

3.2.5 Visualization of the model

For this task we created 2 plots to understand the data easier.

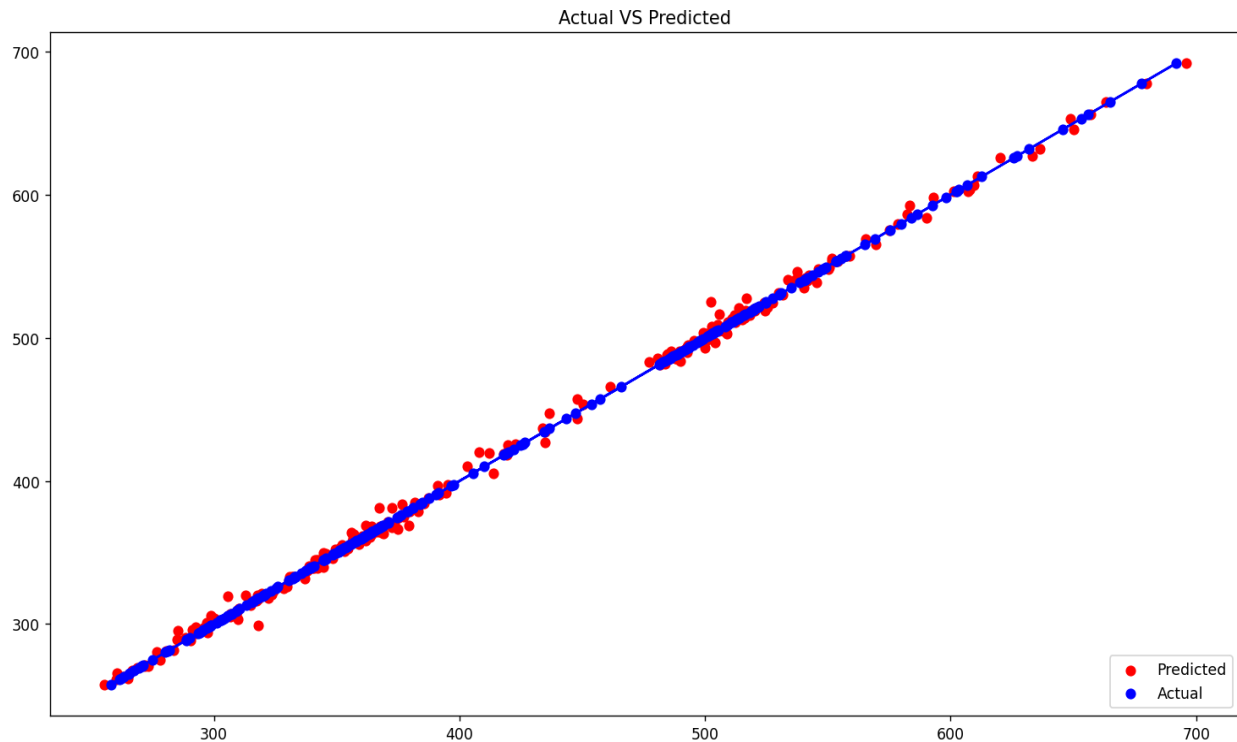
```
viz['Date']=pd.to_datetime(viz['Date'],format='%Y-%m-%d')

plt.title('Closing Stock Price', color="black")
plt.plot(viz.Date, viz.Close, color="red")
plt.legend(["Close"], loc ="lower right", facecolor='white', labelcolor='black')
plt.show()
```



First one is the **plot of closing stock prices over time** with a title, legend, and visually appealing color choices.


```
plt.title('Actual VS Predicted', color="black")
plt.scatter(y_pred, y_test, color='red', marker='o', label='Predicted')
plt.scatter(y_test, y_test, color='blue', label='Actual')
plt.plot(y_test, y_test, color='blue')
plt.legend(loc="lower right", facecolor='white', labelcolor='black')
plt.show()
```



The second one is a **scatter plot to visualize the relationship between the predicted values and the actual values from a regression model**. It also includes a diagonal line representing perfect prediction.

3.2.6 Making a prediction

Here we perform the following steps:

- Making predictions on historical records using a linear regression model.
- Creating a dataframe that stores both the actual and predicted values.
- Adding a 'Date' column to the dataframe.
- Setting the 'Date' column as an index, making it easier to work with the dataframe based on time.

```
# Prediction
close=model_lnr.predict(x)
result=pd.DataFrame({"actual":df["Close"],"predicted":close})
result["Date"]=viz["Date"]
result.set_index(["Date"],inplace=True)
```

3.2.7 Saving the data as CSV

Saving data as a CSV file is a good practice for most data analysis and machine learning tasks because of its universal compatibility, simplicity, and ease of use. It's a convenient format for data exchange, storage, and manipulation, making it a popular choice in the field.

```
# Saving to CSV
result.to_csv("output_stock.csv")
```

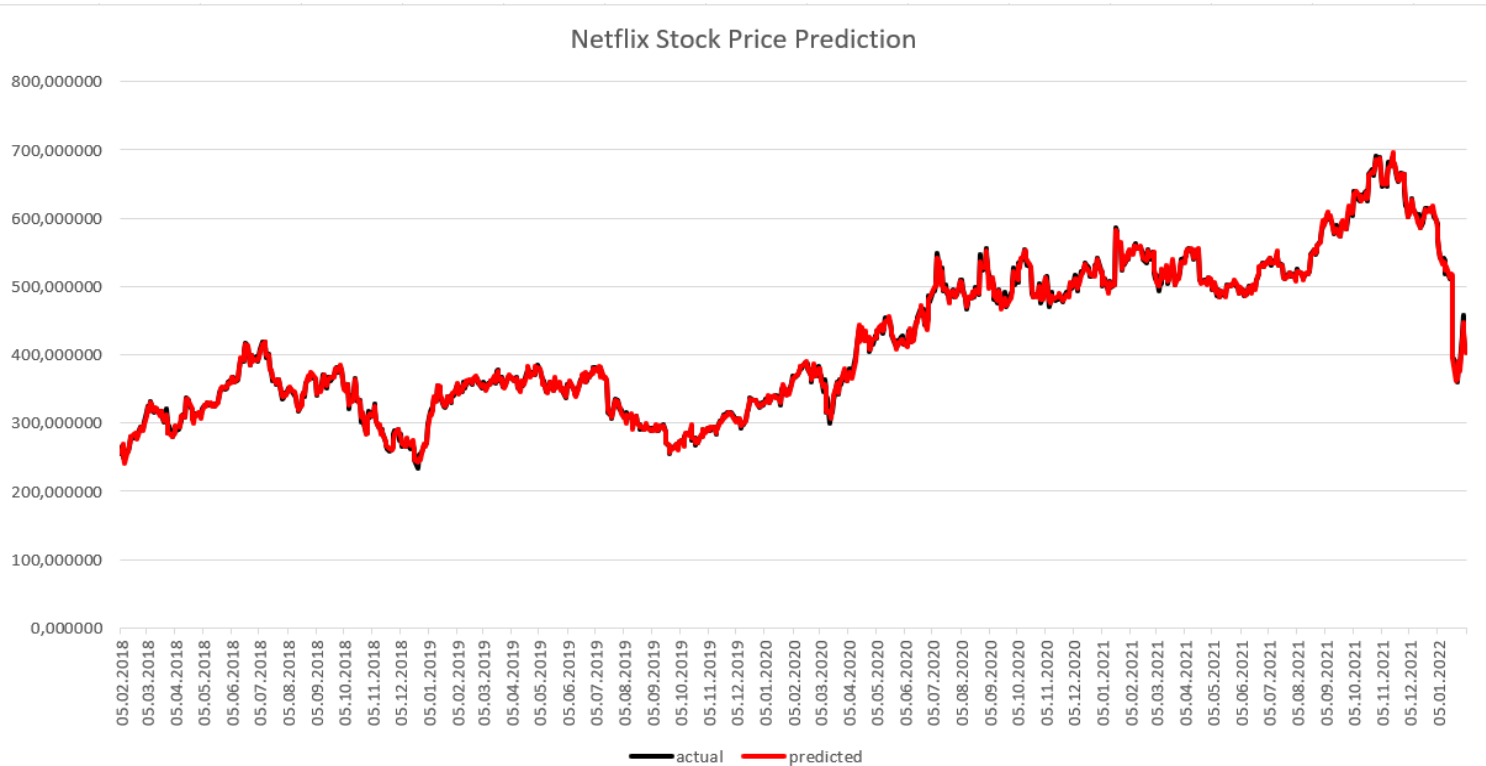
3.2.8 Saved result:

We got a CSV file, where we can see the actual and predicted price at a certain date.

	A	B	C
1	Date ▾	actual ▾	predicted ▾
2	05.02.2018	254,26	257,0972
3	06.02.2018	265,72	260,51869
4	07.02.2018	264,56	269,72146
5	08.02.2018	250,10001	253,57173
6	09.02.2018	249,47	241,53128
7	12.02.2018	257,95001	255,29158
8	13.02.2018	258,26999	258,6149
9	14.02.2018	266	268,23117
10	15.02.2018	280,26999	276,63871
11	16.02.2018	278,51999	279,19309
12	20.02.2018	278,54999	283,38205
13	21.02.2018	281,04001	284,47859
14	22.02.2018	278,14002	276,98893
15	23.02.2018	285,92999	282,55592
16	26.02.2018	294,16	293,26441
17	27.02.2018	290,60999	293,91966
18	28.02.2018	291,38001	293,71589
19	01.03.2018	290,39002	288,00552
20	02.03.2018	301,04999	296,88108
21	05.03.2018	315	310,54421
22	06.03.2018	325,22	323,11571
23	07.03.2018	321,16	319,71329
24	08.03.2018	317	316,86615
25	09.03.2018	331,44	329,25466
26	12.03.2018	321,29999	322,98967

3.2.9 Resulting graph:

On this graph we can see how the Netflix stock prices have changed between February 2018 to February 2022. The black line represents actual prices and the red one – predicted. It is easy to notice that the lines almost overlap each other. It means that our model is quite accurate and predicted prices are almost equal to actual ones.



3.3 Code

```
3     # Import libraries
4     import numpy as np
5     import pandas as pd
6     import matplotlib.pyplot as plt
7     from sklearn.model_selection import train_test_split
8     from sklearn.linear_model import LinearRegression
9     from sklearn.metrics import (mean_squared_error, mean_absolute_error,
10                                  mean_absolute_percentage_error, r2_score)
11
12     # Load the dataset
13     df = pd.read_csv("dataset.csv")
14     # Check dataset information
15     print(df.info())
16     # Check top five rows
17     print(df.head())
18     # Check bottom five rows
19     print(df.tail())
20     # Create deep copy object (the original object will not be effected)
21     viz = df.copy()
22
23     # Check missing values
24     print(df.isna().sum())
25
26     # Split the data
27     x = df.drop(["Close", "Date", "Adj Close"], axis=1)
28     y = df["Close"]
29     x_train, x_test, y_train, y_test = train_test_split(x, y, random_state=0)
30     print(x_train.head())
31
32     # Linear Regression
33     model_lnr = LinearRegression()
34     model_lnr.fit(x_train, y_train)
35     y_pred = model_lnr.predict(x_test)
36
37     # Model Evaluation
38     print("MSE",round(mean_squared_error(y_test,y_pred), 3))
39     print("RMSE",round(np.sqrt(mean_squared_error(y_test,y_pred)), 3))
40     print("MAE",round(mean_absolute_error(y_test,y_pred), 3))
41     print("MAPE",round(mean_absolute_percentage_error(y_test,y_pred), 3))
42     print("R2 Score : ", round(r2_score(y_test,y_pred), 3))
43
44     # Visualization
45     viz['Date']=pd.to_datetime(viz['Date'],format='%Y-%m-%d')
```

```

46
47 plt.title('Closing Stock Price', color="black")
48 plt.plot(viz.Date, viz.Close, color="red")
49 plt.legend(["Close"], loc ="lower right", facecolor='white',
    labelcolor='black')
50 plt.show()
51
52 plt.title('Actual VS Predicted', color="black")
53 plt.scatter(y_pred, y_test, color='red', marker='o', label='Predicted')
54 plt.scatter(y_test, y_test, color='blue', label='Actual')
55 plt.plot(y_test, y_test, color='blue')
56 plt.legend(loc="lower right", facecolor='white', labelcolor='black')
57 plt.show()
58
59 # Prediction
60 close=model_lnr.predict(x)
61 result=pd.DataFrame({"actual":df["Close"],"predicted":close})
62 result["Date"]=viz["Date"]
63 result.set_index(["Date"],inplace=True)
64
65 # Saving to CSV
66 result.to_csv("output_stock.csv")

```

4 Conclusion

This project has explored the potential of using machine learning to predict Netflix's stock price. By applying advanced algorithms like linear regression, we were able **to achieve a 0.998 R2 Score and 0.8% MAPE**, demonstrating the potential of Big Data for financial decision-making.

Thanks to this project we have learned how to prepare and process large datasets, train models using machine learning algorithms and analyze visualized results. It is also worth noting that after training our model we could make successful predictions.

While our model has shown promise, it's essential to acknowledge the complexities of the stock market. Many factors and unexpected events can significantly impact stock prices. Future work could explore more sophisticated models and use real-time data that influence Netflix's stock performance.